Step 1 — The Report Shape and the Compiler — Solution

> Back to the step · Hint 1 · Hint 2

Story: the typed shapes and the deterministic compiler/formatter — built and proven with hand-made parts, no model. **If not:** if the formatting is wrong, perfect extractions still print a messy report, and you'd be debugging two things at once in Step 3.

Setup files

`requirements.txt`: `langchain-core`, `langchain-openai`, `openai`, `pydantic`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
```

```
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes

```
# src/schemas.py
# The typed shapes the extractors fill in and the compiler folds together.
# ActionItems and Decisions wrap a list so with_structured_output has a single
# object to return.
```

```
from typing import List, Optional

from pydantic import BaseModel

class ActionItem(BaseModel):
    task: str
    owner: Optional[str] = None

class ActionItems(BaseModel):
    items: List[ActionItem] = []

class Decisions(BaseModel):
    decisions: List[str] = []

class MeetingReport(BaseModel):
    summary: str
    decisions: List[str] = []
    action_items: List[ActionItem] = []
```

The deterministic compiler and formatter

```
# src/report.py
# Fold the three extractor results into one MeetingReport, and format it for a
# human. Plain Python, so it's fully testable with hand-made parts - no model.
from src.schemas import MeetingReport

def compile_report(parts):
    return MeetingReport(
        summary=parts["summary"],
        decisions=parts["decisions"].decisions,
        action_items=parts["actions"].items,
    )

def format_report(report):
    lines = ["Summary:", "  " + report.summary, "", "Decisions:"]
    for decision in report.decisions:
        lines.append("  - " + decision)
    lines.append("")
    lines.append("Action items:")
    for item in report.action_items:
        owner = "unassigned"
        if item.owner:
            owner = item.owner
```

```
        lines.append(f"  - {item.task} ({owner})")
    return "\n".join(lines)
```

The tests (no model, no key)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MEETINGMIND_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_report.py
# Run from the repo root: python -m tests.unit.test_report
from src.report import compile_report, format_report
from src.schemas import ActionItem, ActionItems, Decisions

def sample_parts():
    return {
        "summary": "Agreed to ship v2 and retire the legacy API.",
        "decisions": Decisions(decisions=[
```

```
            "Ship v2 on Friday", "Retire the legacy API"]),
        "actions": ActionItems(items=[
            ActionItem(task="Send the Q3 budget", owner="Sara"),
            ActionItem(task="Write the migration guide"),
        ]),
    }

def test_report_lists_decisions_and_actions():
    text = format_report(compile_report(sample_parts()))
    result = "OK"
    if "Ship v2 on Friday" not in text:
        result = "WRONG"
    if "Send the Q3 budget (Sara)" not in text:
        result = "WRONG"
    print(f"case 1 (report lists both)        -> {result}")

def test_unassigned_action_reads_unassigned():
    text = format_report(compile_report(sample_parts()))
    result = "OK"
    if "Write the migration guide (unassigned)" not in text:
        result = "WRONG"
    print(f"case 2 (no owner -> unassigned)   -> {result}")

if __name__ == "__main__":
    test_report_lists_decisions_and_actions()
    test_unassigned_action_reads_unassigned()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (report lists both)        -> OK
case 2 (no owner -> unassigned)   -> OK
```

The whole report format is proven before the model exists — so Step 3 only has to feed it real extractions, not debug the layout.